

LAPORAN PRAKTIKUM PEKAN 2
PEMOGRAMAN BERORIENTASI OBJEK
Relasi Antar Objek (Class Diagram & Asosiasi)



Oleh:
Naysha Aprilia Rizki (2511531014)
Kelas B

Dosen Pengampu:
Nurfiah, S.ST., M.Kom.

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG
2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 2 mata kuliah Pemrograman Berorientasi Objek dengan materi Relasi Antar Objek (Class Diagram & Asosiasi) dengan baik.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan. Melalui praktikum ini, penulis mempelajari konsep dasar pemrograman berorientasi objek, khususnya mengenai Relasi Antar Objek (Class Diagram & Asosiasi), serta penerapannya dalam membangun program sederhana berupa Transaksi Debit dan Kredit. Praktikum ini juga membantu penulis memahami bagaimana objek dapat saling berinteraksi dan bagaimana state suatu objek dapat dimanipulasi melalui method.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dalam penyusunan maupun penyajian materi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan evaluasi dan perbaikan untuk laporan maupun praktikum selanjutnya.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai konsep dasar Pemrograman Berorientasi Objek, khususnya dalam Relasi Antar Objek (Class Diagram & Asosiasi).

Padang, 14 September 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman Berorientasi Objek (PBO) merupakan paradigma pemrograman yang menggunakan objek sebagai dasar dalam membangun suatu program. Setelah memahami konsep dasar seperti class, object, constructor, dan method, pemahaman mengenai hubungan atau relasi antarobjek menjadi salah satu hal penting dalam pengembangan program berbasis objek. Relasi antarobjek memungkinkan beberapa objek dalam suatu sistem untuk saling berhubungan dan berinteraksi sesuai dengan fungsi masing-masing.

Salah satu cara untuk menggambarkan relasi antarobjek adalah menggunakan Class Diagram. Class Diagram merupakan bagian dari Unified Modeling Language (UML) yang digunakan untuk menggambarkan struktur suatu sistem, seperti class, atribut, method, serta hubungan yang terdapat antarclass. Dengan adanya Class Diagram, struktur dan hubungan antarobjek dalam suatu program dapat dirancang dan dipahami dengan lebih jelas sebelum program diimplementasikan.

Salah satu bentuk relasi yang umum digunakan dalam Pemrograman Berorientasi Objek adalah asosiasi. Asosiasi menunjukkan adanya hubungan antara satu class dengan class lainnya yang memungkinkan objek dari kedua class tersebut saling berinteraksi. Penerapan asosiasi dapat ditemukan dalam berbagai sistem, salah satunya pada sistem perbankan yang memiliki hubungan antara data rekening dengan transaksi yang dilakukan.

Pada praktikum ini, konsep Relasi Antar Objek diterapkan melalui pembuatan program sederhana berupa Transaksi Debit dan Kredit. Melalui program tersebut, mahasiswa dapat memahami bagaimana beberapa class saling berhubungan, bagaimana objek berinteraksi melalui method, serta bagaimana perubahan state pada suatu objek dapat terjadi akibat interaksi dengan objek lainnya.

Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami konsep Relasi Antar Objek, khususnya Class Diagram dan Asosiasi, serta mampu merancang dan mengimplementasikan hubungan antarclass dalam bahasa pemrograman Java melalui program sederhana Transaksi Debit dan Kredit.

1.2 Tujuan

1. Membaca dan menerjemahkan Class Diagram ke dalam kode program java.
2. Memahami dan mengimplementasikan relasi Asosiasi (one to many)
3. Memanajemen koleksi objek di dalam objek lain menggunakan tipe data bentukan dan struktur dinamis (ArrayList)

1.3 Manfaat

1. Menambah pemahaman mengenai cara membaca dan menerjemahkan Class Diagram ke dalam kode program Java.
2. Meningkatkan kemampuan dalam memahami dan menerapkan relasi Asosiasi (one to many) antarobjek dalam Pemrograman Berorientasi Objek.
3. Memahami cara mengelola dan memanajemen kumpulan objek di dalam objek lain menggunakan tipe data bentukan dan struktur dinamis ArrayList.
4. Membantu mahasiswa dalam memahami hubungan dan interaksi antarobjek sehingga dapat membuat program yang lebih terstruktur dan mudah dikembangkan.
5. Meningkatkan kemampuan mahasiswa dalam menerapkan konsep Pemrograman Berorientasi Objek melalui pembuatan program sederhana menggunakan bahasa pemrograman Java

BAB II

PEMBAHASAN

2.1 Teori

Dalam Pemrograman Berorientasi Objek (PBO), sebuah program tidak hanya terdiri dari satu objek, tetapi dapat memiliki banyak objek yang saling berhubungan dan berinteraksi. Hubungan antarobjek tersebut digunakan untuk menggambarkan bagaimana suatu objek dapat berkomunikasi atau bekerja sama dengan objek lainnya. Untuk mempermudah perancangan hubungan tersebut, digunakan Class Diagram dan konsep asosiasi.

- **Class Diagram**, merupakan salah satu diagram dalam Unified Modeling Language (UML) yang digunakan untuk menggambarkan struktur suatu sistem. Class Diagram dapat menunjukkan class yang terdapat dalam program beserta atribut, method, dan hubungan antarclass. Dengan adanya Class Diagram, struktur program dapat dirancang terlebih dahulu sehingga proses implementasi ke dalam kode Java menjadi lebih terarah.

- **Relasi Antar Objek**, merupakan hubungan yang terjadi antara objek dari suatu class dengan objek dari class lainnya. Relasi ini memungkinkan objek untuk saling berinteraksi dalam menjalankan fungsi tertentu. Dalam sebuah sistem, relasi antarobjek diperlukan agar setiap objek dapat menjalankan perannya dan bekerja sama untuk menghasilkan suatu proses.

- **Asosiasi**, merupakan salah satu bentuk relasi antarclass yang menunjukkan adanya hubungan antara satu class dengan class lainnya. Asosiasi dapat digunakan ketika suatu objek membutuhkan atau memiliki hubungan dengan objek lain. Salah satu bentuk asosiasi adalah **one to many (satu ke banyak)**, yaitu ketika satu objek dari suatu class dapat memiliki hubungan dengan banyak objek dari class lainnya.

- **Asosiasi One to Many**, dapat diterapkan pada sistem transaksi perbankan. Sebagai contoh, satu objek **Rekening** dapat memiliki banyak objek **Transaksi**. Satu rekening dapat digunakan untuk melakukan beberapa transaksi, baik transaksi debit maupun kredit. Hubungan tersebut dapat digambarkan dalam Class Diagram dan kemudian diterapkan ke dalam kode program Java.

- **ArrayList**, merupakan salah satu struktur data dinamis dalam Java yang digunakan untuk menyimpan sekumpulan objek. ArrayList memiliki ukuran yang dapat bertambah atau berkurang sesuai dengan jumlah data yang disimpan. Dalam penerapan asosiasi one to many, ArrayList dapat digunakan untuk menyimpan banyak objek dari suatu class di dalam objek dari class lainnya. Contohnya, objek Rekening dapat memiliki sebuah ArrayList yang digunakan untuk menyimpan kumpulan objek Transaksi.

Dalam konteks program **Transaksi Debit dan Kredit**, class Rekening dapat digunakan untuk menyimpan informasi rekening, sedangkan class Transaksi digunakan untuk menyimpan informasi transaksi yang dilakukan. Hubungan antara keduanya menggunakan asosiasi one to many, karena satu rekening dapat memiliki banyak transaksi. Kumpulan transaksi tersebut dapat dikelola menggunakan ArrayList sehingga data transaksi dapat ditambahkan dan dikelola secara dinamis.

2.2 Langkah Langkah

1. Membuat Class Transaksi

- 1) Pertama saya membuat Package “Prak2_Relasi”
- 2) Selanjutnya dilanjutkan dengan membuat Class Transaksi, Berikut Screenshot lengkap nya

```
1 package Prak2_Relasi;
2
3 public class Transaksi {
4     String idTransaksi ;
5     String jenis;
6     double nominal;
7
8     //constructor
9     public Transaksi(String id, String jenis, double nominal) {
10         this.idTransaksi = id;
11         this.jenis = jenis;
12         this.nominal = nominal;
13     }
14     public void cetakDetail () {
15         System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
16     }
17 }
18
```

Penjelasan:

- 1) Buat class Transaksi

```
public class Transaksi {
```

Ini mengartikan kita sedang membuat sebuah class bernama Transaksi. Class bisa dianggap sebagai cetakan/model untuk membuat objek Transaksi.

- 2) Membuat State/Atribut

```
public class Transaksi {
    String idTransaksi ;
    String jenis;
    double nominal;
```

Atribut ini merupakan variable yang menyimpan informasi tentang suatu objek. Pada class transaksi ini tersimpan data berupa

- idTransaksi dengan tipe data string
- jenis dengan tipe data string
- nominal dengan tipe data double.

3) Buat Constructor

```
//constructor
public Transaksi(String id, String jenis, double nominal) {
    this.idTransaksi = id;
    this.jenis = jenis;
    this.nominal = nominal;
}
```

Constructor ini digunakan untuk menginisialisasikan atau memberi nilai awal pada objek Transaksi Ketika objek dibuat.

- idTransaksi di inisialisasikan dengan nilai id
- jenis diinisialisasikan dengan nilai jenis transaksi, yaitu jenis transaksi debit atau kredit
- nominal diinisialisasikan dengan nilai nominal transaksi

4) Buat Method/Behavior

```
public void cetakDetail () {
    System.out.println("ID: " + idTransaksi + " | Jenis: " + jenis + " | Nominal: Rp" + nominal);
}
```

Method cetakDetail() digunakan untuk menampilkan detail transaksi yang berisi ID transaksi, jenis transaksi, dan nominal transaksi. Program akan menampilkan informasi dengan format “ID: (ID transaksi) | Jenis: (jenis transaksi) | Nominal: Rp (nominal)”.

2. Membuat Relasi pada Class Rekening

Salin Code Program yang sudah dibuat pada Prak1_MiniBank, sebagai relasi nya. Dan akan ditambahkan atribut baru yang mempresentasikan relasi.

```
1 package Prak2_Relasi;
2 import java.util.ArrayList;
3
4 public class Rekening {
5     String nomorRekening;
6     String namaPemilik;
7     double saldo;
8
9     //Implementasikan Association (1-to-many)
10    ArrayList<Transaksi> riwayatTransaksi;
11
12    public Rekening(String nomorRekening, String namaPemilik, double saldoawal) {
13        this.nomorRekening = nomorRekening;
14        this.namaPemilik = namaPemilik;
15        this.saldo = saldoawal;
16
17        //untuk menginisialisasikan ArrayList di dalam constructor agar tidak NullPointerException
18        this.riwayatTransaksi = new ArrayList<>();
19
20        System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");
21    }
22
23 }
```

- ArrayList<Transaksi> riwayatTransaksi;

Digunakan untuk menyimpan kumpulan data transaksi yang dilakukan oleh rekening. Jadi, setiap transaksi bisa dicatat dan disimpan dalam riwayatTransaksi.

- **this.riwayatTransaksi = new ArrayList<>();**

Digunakan untuk membuat ArrayList baru yang kosong sebagai tempat menyimpan riwayat transaksi. this menunjukkan bahwa yang digunakan adalah atribut riwayatTransaksi milik objek tersebut.

- **System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat.");**

Digunakan untuk menampilkan pesan ke layar bahwa rekening berhasil dibuat, sekaligus menampilkan nama pemilik rekening.

3. Membuat Riwayat transaksi

Perbaiki method setor tunai dan Tarik tunai di class rekening java agar otomatis menginstansiasi objek transaksi dan menyimpannya ke riwayatTransaksi saat transaksi berhasil.

```
public void setorTunai(double nominal) {  
    if (nominal > 0) {  
        saldo += nominal;  
        //Berkas riwayat (Pembuatan objek Transaksi di dalam method)  
        String idTrx = "TRX-S-" + System.currentTimeMillis();  
        Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);  
        riwayatTransaksi.add(trxBaru);  
  
        System.out.println("Setor tunai Rp" + nominal + " berhasil Saldo saat ini: Rp" + saldo);  
    } else {  
        System.out.println("Gagal: Nominal setor harus lebih dari 0! ");  
    }  
}
```

Method Setor tunai menjelaskan bahwa jika nominal lebih dari 0, maka saldo akan bertambah sesuai nominal yang disetor, kemudian transaksi yang disetor, kemudian transaksi dicatat ke dalam riwayat transaksi dan program akan menampilkan “Setor tunai Rp (jumlah nominal) berhasil, saldo saat ini Rp (jumlah saldo)”. Jika nominal 0 atau kurang, maka program akan menampilkan pesan bahwa setor tunai gagal.

- **String idTrx = "TRX-S-" + System.currentTimeMillis();** ini digunakan untuk membuat ID transaksi otomatis. **TRX-S-** menandakan transaksi setor, sedangkan **currentTimeMillis()** memberikan angka waktu saat transaksi dibuat.
- **Transaksi trxBaru = new Transaksi(idTrx, "Kredit", nominal);** Membuat objek transaksi baru dengan **idTrx**(id Transaksi), **“Kredit”**(Jenis transaksi setor), **nominal**(jumlah uang)
- **riwayatTransaksi.add(trxBaru);** Memasukkan transaksi tersebut ke dalam riwayat transaksi.

Latihan/Tugas

1. Method Tarik Tunai

```
40 }
41 public void tarikTunai(double nominal) {
42     String idTrx = "TRX-T-" + System.currentTimeMillis();
43     Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);
44     riwayatTransaksi.add(trxBaru);
45
46     if (nominal < 10000) {
47         System.out.println("Transaksi Gagal: Minimal Penarikan 10.000");
48     } else if (nominal > saldo) {
49         System.out.println("Transaksi Gagal saldo tidak mencukupi. Saldo Anda: Rp" + saldo);
50     }
51     else {
52         saldo -= nominal;
53         System.out.println("Transaksi Berhasil");
54     }
55 }
```

Pada Class Rekening, pada bagian method Tarik tunai ubah beberapa bagian seperti syarat dari tugas yaitu Tambahkan logika perekaman objek Transaksi seperti pada contoh setorTunai. Pastikan jenis transaksinya dicatat sebagai "Debit" dan kodenya menggunakan prefiks "TRX-T-". Transaksi hanya boleh dicatat jika penarikan berhasil (saldo mencukupi).

- **String idTrx = "TRX-T-" + System.currentTimeMillis();** ini digunakan untuk membuat ID transaksi otomatis. **TRX-T** menandakan transaksi tarik, sedangkan **currentTimeMillis()** memberikan angka waktu saat transaksi dibuat.
- **Transaksi trxBaru = new Transaksi(idTrx, "Debit", nominal);** Membuat objek transaksi baru dengan **idTrx**(id Transaksi), **"Debit"**(Jenis transaksi setor), **nominal**(jumlah uang)
- **riwayatTransaksi.add(trxBaru)** Memasukkan transaksi tersebut ke dalam riwayat transaksi.

2. Cetak Mutasi Rekening

1) Tambahkan Method cetakMutasi pada bagian Class Rekening

```
public void cetakMutasi() {
    if (riwayatTransaksi.isEmpty()) {
        System.out.println("Belum ada transaksi pada rekening ini");
    } else {
        for (Transaksi transaksi : riwayatTransaksi) {
            transaksi.cetakDetail();
        }
    }
}
```

Memnambahkan perulangan for-each untuk menulusrui koleksi riwayatTransaksi nya. Jika riwayatTransaksi kosong maka system akan print "Belum ada transaksi pada rekening ini". Selain itu untuk transaksi akan diperiksa Riwayat dan cetak detail nya.

2) Pada Class Main tambahkan while dan juga case 6

```

while (isRunning) {
    System.out.println("\nMenu Utama:");
    System.out.println("1.Buka Rekening Baru");
    System.out.println("2.Setor Tunai");
    System.out.println("3.Tarik tunai");
    System.out.println("4.Cek Informasi Rekening");
    System.out.println("5.Ganti Akun");
    System.out.println("6.Cetak Mutasi (Riwayat)");
    System.out.println("0.Keluar");
    System.out.println("Pilih Menu: ");
}

```

Tambahkan pada bagian 6.Cetak Mutasi (Riwayat)

```

case 6:
    akunAktif.cetakMutasi();
    break;

```

Menambahkan case ke 6 yang digunakan untuk menjalankan method cetakMutasi() pada akun yang sedang aktif untuk menampilkan Riwayat transaksi yang telah dilakukan dan break untuk keluar dari case tersebut.

OUTPUT

```

=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 001
Masukan Nama Pemilik: Hry
Masukan Saldo Awal: 500.000
Rekening atas nama Hry berhasil dibuat.

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
2
Masukan nominal setor: 50.000
Setor tunai Rp50000.0 berhasil Saldo saat ini: Rp350000.0

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
3
Masukan nominal tarik: 150.000
Transaksi Berhasil

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
6.Cetak Mutasi (Riwayat)
0.Keluar
Pilih Menu:
0
ID: TRX-5-1789820151753 | Jenis: Kredit |Nominal: Rp50000.0
ID: TRX-7-1789820186017 | Jenis: Debit |Nominal: Rp150000.0

```

Buka rekening baru dan lanjutkan dengan setor tunai dan Tarik tunai. Dan lanjutkan dengan cetak mutasi atau Riwayat.

